

Week 5: Transform 与语义层——把口径写进工程（给 BI 也给 Agent 用）

把“业务指标怎么算”从 SQL 片段升级成工程接口

Week05 的重点不是多写几个漂亮 SQL，也不是把课堂变成 dbt Cloud 或语义层产品演示。

这一周只解决一个核心问题：

指标不是 SQL 结果，而是 BI、运营、Agent、评测和治理共同承认的工程接口。

本周实践对齐 `dataPro-lgtm/omnisupport-copilot` 当前 Week05 实现：`analytics/dbt Core` 工程、`analytics/metric_registry_v1.yml`、`contracts/tools/tools/query_support_kpis_v1.json`、`services/tool_api/app/kpi_query.py`、`runbooks/week05/README.md` 与 `reports/week05/dbt_build_evidence.md`。课程页只讲站点内容，不修改项目代码。

本周一句话

把 Week04 的“有状态表”推进成 有口径、有测试、有文档、有血缘、有工具边界 的 analytics engineering 基线。

本周为什么重要

Week04 已经让表状态可以被回看和绑定 release。到了 Week05，风险从“数据状态说不清”转成“同一个业务指标到底怎么算”。

- 如果指标只藏在 dashboard SQL 里，BI、运营、Agent 很快会各算各的。
- 如果 transform 没有 tests / docs / lineage，口径变更会变成隐性事故。
- 如果 Agent 直接写 SQL，权限、成本、审计和口径一致性都会失控。
- 如果没有 metric registry，工具 runtime 只能猜哪些指标、维度、过滤器是安全的。

先用一个真实问题理解 Week5

运营经理问了一个很常见的问题：

“最近 7 天 P1 工单是不是明显变多了？”

如果这个问题没有被工程化，三个系统很可能给出三个答案。

回答者	它可能怎么查	得到的数字为什么不同	真正暴露的问题
运营看板	用 dashboard SQL 按 <code>created_at</code> 聚合 P1 工单	只算新建工单，不一定算 reopen 或升级后的 P1	时间字段和业务状态没有约定
BI 临时报表	重新写一条 SQL，按 <code>updated_at</code> 或工单状态变更时间算	把历史工单重新升级为 P1 的情况也算进来	粒度、过滤条件、口径窗口没有约定

回答者	它可能怎么查	得到的数字为什么不同	真正暴露的问题
Agent 工具	直接查底表或让模型生成 SQL	可能把正文、PII、非授权维度也带出来	工具边界和审计链路没有约定

这不是“谁 SQL 写得更好”的问题，而是 **P1 工单增长** 还没有成为一个工程接口：谁拥有它、按什么粒度算、从哪里来、允许哪些维度、哪些角色能查、怎么测试、怎么审计，都没有被写下来。

! Week05 不等于 SQL 课，也不等于 dbt Cloud 产品课

本周真正训练的是：如何让口径成为可测试、可文档化、可审计、可被工具消费的工程接口。dbt Core、metric registry 和 tool contract 只是把这个接口落地的最小组合。

Week05 做什么 / 不做什么

本周做

- 用 dbt Core 把 source 到 mart 的 transform 做成可复现工程
- 建立 `support_case_mart`、`support_kpi_mart`、`agent_tool_input_view`
- 用 tests / docs / lineage 证明口径能负责
- 用 `metric_registry_v1.yml` 固定指标、维度、过滤器和角色边界
- 用 `query_support_kpis_v1` 让 Agent 查询受控指标，而不是裸写 SQL

本周不做

- 不把 dbt Cloud 或托管 Semantic Layer 做成必修依赖
- 不把 MetricFlow 生产化作为 Student Core
- 不做 NL2SQL，也不允许 Agent 任意查库¹
- 不重写 Week04 lakehouse 和 Week06 orchestration
- 不承诺 Week08 / Week10 才会展开的检索和工具治理能力

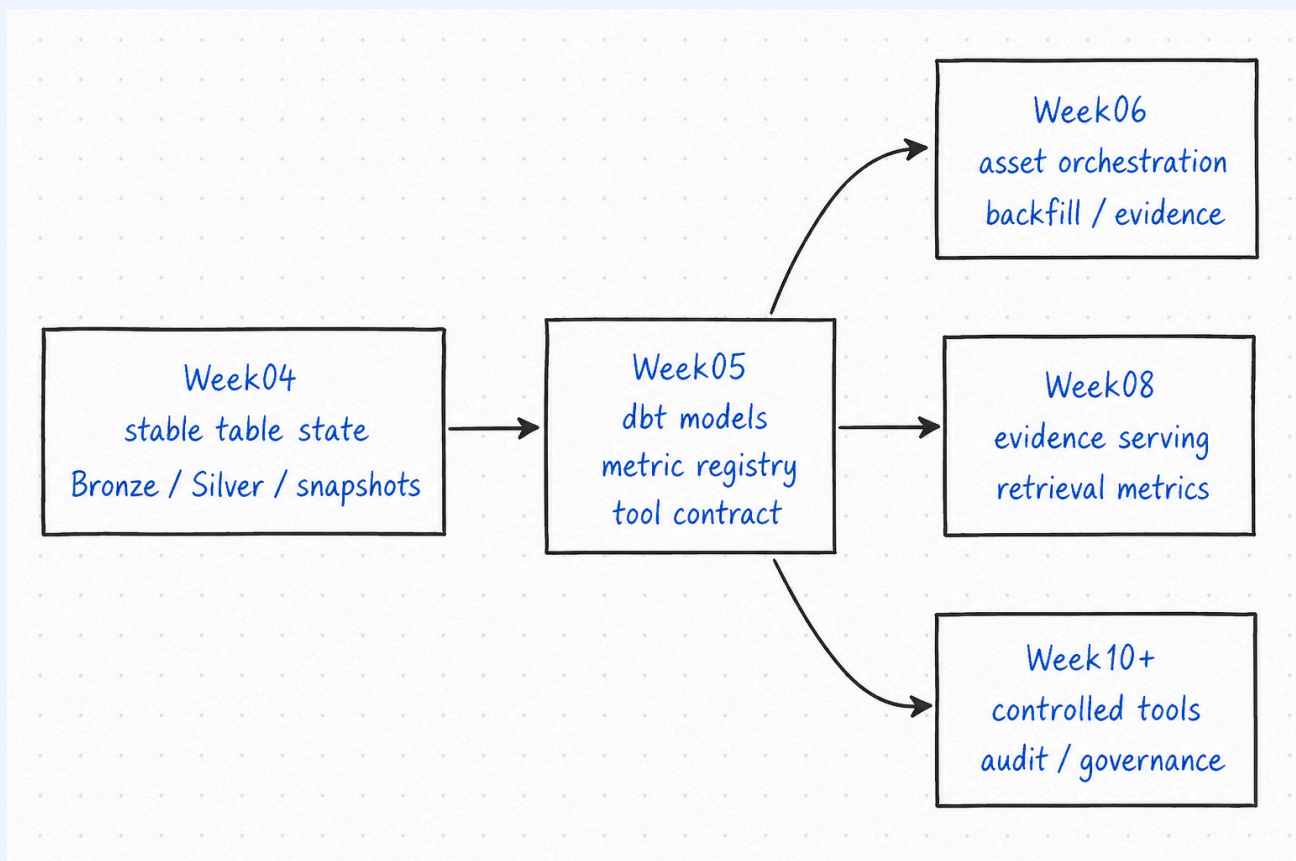
Week5 的内涵和外延

概念	小白版解释	本周怎么落	后续谁会用
dbt transform	把“怎么从原始表变成业务表”写成可复现工程	<code>analytics/</code> 里的 <code>sources</code> 、 <code>staging</code> 、 <code>intermediate</code> 、 <code>mart</code> s	数据团队、BI、Week06 asset 编排
指标口径	同一个指标到底怎么算、按什么粒度算	从 <code>support_case_mart</code> 到 <code>support_kpi_mart</code> 固定 P1、SLA、reopen、escalation 等口径	BI、运营复盘、Agent、评测
tests	用自动检查证明口径没有明显破坏	dbt tests 与 <code>tests/contract/test_week05_metric_contract.py</code>	回归、上线门禁、课程验收
docs	给未来使用者解释模型和字段含义	dbt docs generate 与 <code>reports/week05/dbt_build_evidence.md</code>	运营、讲师、后续维护者

¹ 本周不做 NL2SQL，因为“让模型生成 SQL”不能自动解决口径、权限、成本、审计和可复现问题。先把可控指标接口做稳，后续工具治理才有边界。

概念	小白版解释	本周怎么落	后续谁会用
lineage	说明一个指标从哪些上游表和模型推出来	dbt DAG、artifacts 与 impact notes	变更影响分析、事故复盘
metric registry	指标、维度、过滤器、角色和窗口的白名单	analytics/metric_registry_v1.yml	Tool runtime、Agent、审计
tool contract	Agent 能怎么问、不能怎么问、输出必须长什么样	contracts/tools/tools/query_support_kpis_v1.json	Week10 工具治理、Agent 调用、评测
语义层外延	让指标被更多系统用同一种含义消费	本周先做本地 registry，不强依赖托管平台	BI、Agent、eval、audit、后续治理
审计外延	让每次指标查询都有边界和证据	工具输出、拒绝码、audit 字段、query examples	安全、合规、事故复盘

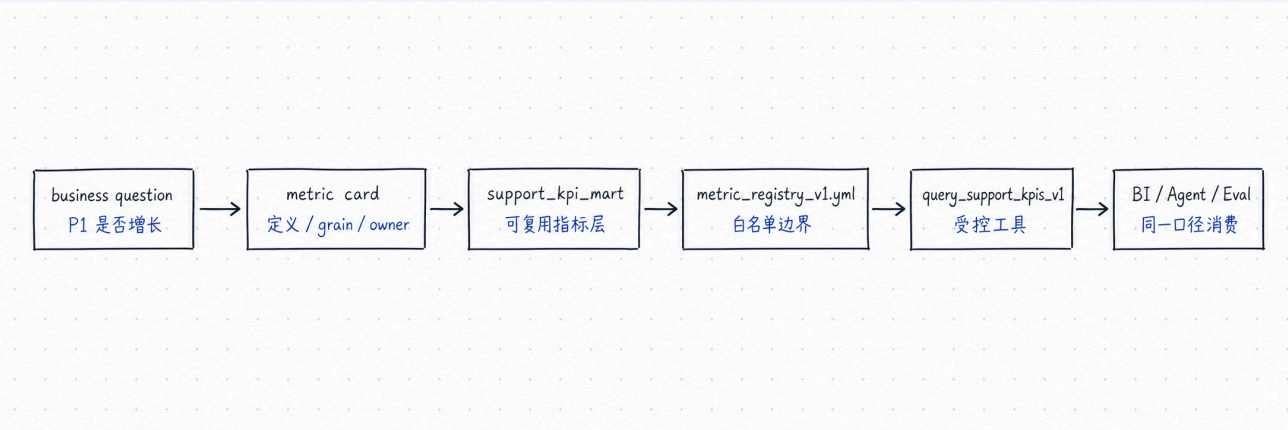
Week04 → Week05 → Week06 / Week08 / Week10 主线



这条线要说明：

- Week04 解决表状态能不能复盘；
- Week05 解决口径能不能复用；
- Week06 解决这些资产能不能被编排、回填和交接；
- Week08 和 Week10 继续消费 Week05 的指标与工具边界。

本周学习路线：从口径卡片到受控工具



节点	这一关要做的工程判断
业务问题	先确认问题是不是一个稳定指标, 而不是临时 SQL 请求
dbt models	把口径从 source 一层层长出来, 避免每个 dash-board 复制一套逻辑
tests / docs / lineage	证明模型不是“能跑就行”, 而是可以解释、可以变更、可以追责
metric registry	把指标、维度、过滤器、角色和时间窗口写成 runtime 能执行的白名单
query_support_kpis_v1	Agent 只能通过契约查指标, 不能绕过白名单直接写 SQL
BI / Agent / Eval	不同消费方拿到同一口径, 后续评测和审计才有共同基线

💡 建议阅读顺序

先读本周总览确认边界, 再按课时 1 到课时 5 走完“指标接口 → dbt 分层 → 证据链 → registry → 工具契约”。实验页负责跑通最小闭环, 作业页负责把闭环整理成可交付资产包。

本周在项目里的真实锚点

课程对象	omnisupport-copilot 路径	本周怎么用
dbt project	analytics/	放 sources、staging、intermediate、marts、tests、docs
source 声明	analytics/models/sources.yml	使用 omni_postgres source, 不让下游模型猜表
staging	analytics/models/staging/stg_*.sql	统一字段、类型、枚举和 PII 边界
intermediate	analytics/models/intermediate/int_*.sql	组合 case 与 daily activity, 不直接暴露给 Agent

课程对象	omnisupport-copilot 路径	本周怎么用
marts	analytics/models/marts/ support_case_mart.sql、 support_kpi_mart.sql	形成 case 粒度和 metric 粒度消费层
safe view	analytics/models/marts/ agent_tool_input_view.sql	工具 runtime 只读安全视图
metric registry	analytics/ metric_registry_v1.yml	固定 6 个指标、4 个维度、过滤器、角色和 31 天窗口
tool contract	contracts/tools/tools/ query_support_kpis_v1.json	定义输入输出 schema、拒绝码和审计字段
tool runtime	services/tool_api/app/ kpi_query.py	校验 registry，生成确定性参数化查询
runbook	runbooks/week05/README.md	实验页命令以它为准
build evidence	reports/ week05/dbt_build_evidence.md	记录 dbt build、docs、registry、tool 和 pytest 证据

五个课时怎么连起来

课时 1 | 先改掉“指标 = SQL”的直觉

用 OmniSupport 的 P1 工单、SLA、reopen、escalation 场景说明：同名指标如果没有 owner、grain、source、filters、tests、docs 和 audit，就不是工程接口。

[进入课时 1](#)

课时 2 | 把口径从 source 一层层长出来

讲清 dbt 是把 SQL transform 变成有目录、有依赖、有测试、有文档、有可复现命令的工程，不是数据库、BI 工具或采集工具。

[进入课时 2](#)

课时 3 | 证明口径能负责

model 能跑出来不等于可以交付。要用 tests、docs、lineage、artifacts 和 impact notes 给未来消费者留下证据。

[进入课时 3](#)

课时 4 | 把语义层讲成现实边界

解释 semantic model、entity、dimension、measure、metric、semantic graph；同时说明本周为什么先落本地 registry。

[进入课时 4](#)

课时 5 | 把指标交给 Agent，但不让 Agent 裸写 SQL

把 metric registry、JSON Schema、角色过滤、时间窗口、防注入、审计和负例测试串成 query_support_kpis_v1。

[进入课时 5](#)

本周交付物保护什么问题

交付物	它保护的不仅是文件，而是...	如果缺失	下游谁会用
<code>support_case_mart</code>	case 粒度稳定，不让每个指标重复解释 ticket 状态	P1、SLA、reopen 会被各自重新理解	BI / Ops / KPI mart
<code>support_kpi_mart</code>	指标聚合口径一致，避免每个 dashboard 自己写一版	同名指标在不同页面出现不同数字	BI / Agent tool / eval
<code>agent_tool_input_view</code>	工具层只读安全字段，不接触 PII 或正文	Agent 查询会越过数据边界	Tool runtime
<code>metric_registry_v1.yml</code>	名称、维度、过滤器、角色、时间窗口白名单	runtime 不知道哪些问题该拒绝	Tool runtime / data team
<code>query_support_kpis_v1</code>	Agent 查询入口受控、可审计、可拒绝	模型可能裸写 SQL、越权、跑无界查询	Agent / audit
dbt docs / lineage	口径解释和变更影响可以被追溯	字段改了以后没人知道影响哪些指标	Data team / instructor
<code>dbt_build_evidence.md</code>	build、tests、docs、registry、tool 正负例都留下证据	“我跑过”无法被验收和复盘	课程验收 / 回归复盘

小白最容易误解的地方

你可能以为	实际上
指标就是一条 SQL	指标是定义、来源、粒度、测试、文档、权限和接口的组合
dashboard 能显示就够了	Agent、评测、治理和运营复盘也需要同一口径
SQL 改了大家同步一下就行	没有 lineage 和 docs，影响范围不可控
Agent 会自动写对 SQL	Agent 很可能写出能运行但口径错误、越权或不可审计的 SQL
语义层等于买一个平台	Week05 先训练本地可跑、可解释、可迁移的最小 semantic contract

学完本周你应该带走的 7 个判断

1. 指标不是 SQL 片段，而是可以被多个系统共同消费的工程接口。
2. grain 比 SQL 花样更重要；粒度错了，聚合再漂亮也不可信。
3. tests、docs、lineage 不是文档负担，而是口径能负责的证据链。
4. 语义层不是先买平台，而是先让团队对指标含义达成可执行约定。²
5. Agent 可以查询指标，但不能越过 tool contract 直接查库。³

² dbt Semantic Layer 可以把 semantic models、entities、dimensions、measures、metrics 组织成平台能力；Week05 先训练本地可执行的语义契约，不把托管平台作为学生硬依赖。

6. Student Core 先跑通本地可迁移的 `metric_registry_v1.yml`，不要把 MetricFlow 生产化当成本周硬依赖。⁴
7. Week06、Week08、Week10 会继续消费 Week05 的 models、registry 和工具边界。

实验与作业的关系

实验页要求你跑通最小闭环：`dbt debug`、`dbt build --select tag:week05`、`dbt docs generate`、registry 校验、KPI 工具正负例、pytest 回归。

作业页要求你把实验结果整理成正式“口径资产包”：ADR、执行蓝图、dbt models、metric registry、tool contract、evidence、examples、delivery summary。

延伸阅读

主题	推荐资料	为什么读
dbt sources	dbt Docs: Sources	理解为什么下游模型不应该直接猜原始表
dbt data tests	dbt Docs: Data tests	理解口径上线前最小质量门禁
dbt documentation	dbt Docs: Documentation	理解 docs 和 lineage 为什么是工程证据
dbt Semantic Layer	dbt Docs: Semantic Layer	对比本周本地 registry 与托管语义层的边界
MetricFlow	dbt Docs: MetricFlow commands	理解语义指标可以如何被 CLI 查询，但本周不要求生产化
OpenAI 工具调用	OpenAI Docs: Function Calling	理解 tool contract 为什么要把输入输出结构写清楚
OpenAI Structured Outputs	OpenAI Docs: Structured Outputs	理解严格 JSON Schema 与可审计工具输出的关系
Quarto 站点写法	Quarto Callouts 、 Mermaid 、 Code Blocks	只作为课程站点实现参考，保持整站样式一致

Footnotes

³Structured Outputs 解决模型输出是否符合 JSON Schema 的问题；tool contract 还要继续约束指标白名单、角色、时间窗口、拒绝码和审计字段。

⁴MetricFlow 是理解语义层的重要参考，但本周 Student Core 的交付重心是 `metric_registry_v1.yml` 与受控 KPI 查询工具，而不是完整 MetricFlow 生产链路。